Computer Methods and Programs in Biomedicine 160 (2018) 95–101

Contents lists available at ScienceDirect

Computer Methods and Programs in Biomedicine

journal homepage: www.elsevier.com/locate/cmpb

A future-proof architecture for telemedicine using loose-coupled modules and HL7 FHIR

Kirstine Rosenbeck Gøeg*, Rune Kongsgaard Rasmussen, Lasse Jensen, Christian Møller Wollesen, Søren Larsen, Louise Bilenberg Pape-Haugaard

Department of Health Science and Technology, Aalborg University, Fr Bajers Vej 7D 9220 Aalborg Ø, Denmark

ARTICLE INFO

Article history:
Received 18 September 2017
Revised 28 December 2017
Accepted 9 March 2018

Keywords:
Telemedicine
IT-architecture
Standardization
HL7 FHIR
Open source

ABSTRACT

Background and objectives: Most telemedicine solutions are proprietary and disease specific which cause a heterogeneous and silo-oriented system landscape with limited interoperability. Solving the interoperability problem would require a strong focus on data integration and standardization in telemedicine infrastructures. Our objective was to suggest a future-proof architecture, that consisted of small loose-coupled modules to allow flexible integration with new and existing services, and the use of international standards to allow high re-usability of modules, and interoperability in the health IT landscape.

Methods: We identified core features of our future-proof architecture as the following (1) To provide extended functionality the system should be designed as a core with modules. Database handling and implementation of security protocols are modules, to improve flexibility compared to other frameworks. (2) To ensure loosely coupled modules the system should implement an inversion of control mechanism. (3) A focus on ease of implementation requires the system should use HL7 FHIR (Fast Interoperable Health Resources) as the primary standard because it is based on web-technologies.

Results: We evaluated the feasibility of our architecture by developing an open source implementation of the system called ORDS. ORDS is written in TypeScript, and makes use of the Express Framework and HL7 FHIR DSTU2. The code is distributed on GitHub. All modules have been tested unit wise, but end-to-end testing awaits our first clinical example implementations.

Conclusions: Our study showed that highly adaptable and yet interoperable core frameworks for telemedicine can be designed and implemented. Future work includes implementation of a clinical use case and evaluation.

1. Introduction

Care and treatment for an aging population with high incidence of chronic disease is a core challenge in modern societies [1]. There is a growing realization that patient empowerment, i.e. patient engagement in their own care, is important in facing that challenge [2]. Among other initiatives, information technology and especially telemedicine are suggested as important technological solutions to improve empowerment and consequently ease the work of health care professionals. Evaluations of telemedicine solutions for patients are common in the scientific literature [3], but transformation of telemedicine into routine care is a challenge [3–5]. Reasons for slow adoption are partially organizational e.g. lack of incentive for health care professionals given the legislation [5]. Partially due to heterogeneity of telemedicine implementations being a challenge. Ad hoc proprietary disease specific systems make development costly, and hinders interoperability with the rest of the health IT landscape.

From an infrastructure perspective, several authors describe that the problem is that modern electronic health record (EHR) and home-health infrastructures are built on top of multiple systems that have a lot of overlapping functionalities. Consequently, they must synchronize data in between systems, and save redundant data in each system [6,7]. In the following we look at some of the research that suggests solutions for these infrastructure challenges. Bridges [7] suggest an approach where it is ensured that system functionality does not overlap, and where data is synchronized between the systems as needed. In contrast, the Danish OpenTele project suggests that re-designing, so that existing and new systems use a single storage, is a better way forward [8]. An American infrastructure project, suggested that “starting over” i.e. implementing new web- and cloud-based technology rather than continue using legacy systems was a more feasible approach [6]. These

* Corresponding author.
E-mail address: kirse@hst.aau.dk (K.R. Gøeg).

https://doi.org/10.1016/j.cmpb.2018.03.010
0169-2607/

examples illustrate, that telemedicine systems contribute to functional and storage overlaps in EHR infrastructures, and that very different practical and scientific solutions are suggested for solving this problem.

Other authors suggest that standardization is a key area for telemedicine [9], and evolutionary change towards generic platforms for multiple conditions are expected in the future [10]. In addition, combining standardization with an open source approach may further decrease costs, and ensure harmonized solutions.

Generic platforms and standardized frameworks have the inherited risk of being so generic that disease specific applications cannot be supported. This would be a major problem for telemedicine because telemedicine solutions are in a transformation from providing information to different stakeholders to being user centric [4], to improve outcome for patients. User centricity means that functionality such as decision support and prediction algorithms to improve patient outcome, as well as bi-directional communication become increasingly common (e.g Melillo et al. suggest build-in decision support [11]). Consequently, future generic platforms need to be highly adaptable to new needs.

New common platforms or standardized frameworks tend to require re-design of existing information systems. However, evolutionary change towards interoperable solutions is often preferred by stakeholders [10], probably because resources cannot be spent on re-design of solutions that already work. We conclude, that when suggesting generic platforms for telemedicine, it is of high important to balance the need for standardization, re-use and interoperability, with the need for highly adaptable solutions which can easily integrate with new functionalities as well as legacy systems.

Adaptability, as a feature in modern web-based systems, is a matter of integrating small and autonomous modules that were not necessarily designed to work together in the first place. This requires a systematic approach to loosely coupling systems together, rather than tight integration.

In accordance with Teijeiro et al [12], we suggest that inversion of control [13] is a key feature in future generic platforms for telemedicine.

The first aim of this study was to suggest a future-proof architecture for telemedicine based on:

- Loose coupling and inversion of control mechanisms to allow flexible integration with new and existing services
- Open source development and the use of international standards to allow high re-usability of system components and interoperability with existing system landscapes

The second aim, was to implement the core of this system to prove feasibility.

The paper is organized as follows: First, we will describe our design considerations applied in the developed architecture through a specification of key features; loose coupling and inversion of control to meet requirements for adaptability, and use of international standards to meet requirements of re-usable software. In Section 3, we will elaborate on the intended use through an explanation of the most important implementation details and what constitute a successful open source strategy. Finally, in Section 4, we discuss our solution compared to the other telemedicine initiatives.

2. Design considerations

In this section, we present our principal design and implementation goals by arguing for and specifying the key features of our future-proof telemedicine architecture namely

- Loose coupling and inversion of control mechanisms
- Use of international standards

2.1. Loose coupling and Inversion of Control (IoC)

To accommodate the needs for adaptability mentioned in the introduction, we wanted to ensure loose coupling of modules and easy integration with legacy systems. In contrast to other frameworks, we wanted our modularization of functionalities to include database technology and security protocols rather than considering these two modules as part of the core. We wanted this high degree of modularization because it means that integration with legacy system becomes easier i.e. the core will not assume that you use a specific database OR security protocol. Consequently, it became a core requirement for us, that it should be possible to integrate our core with components of third-party systems so that e.g. a legacy database with 10 years of diabetes-patient information did not have to be migrated since it could be integrated instead. In conventional systems, this flexibility does not exist, and developers are forced to deal with redundant technologies where middleware must be implemented to bridge the gap between the current implementation and third-party systems. In addition to supporting integration with legacy systems, modularization provides flexibility that ensures re-use of legacy systems and databases when that is applicable, and addition of new modules, when new clinical use cases are implemented.

Yet, this design choice alleviates the burden for the core designers and instead moves it to the module developers. This could give a slow uptake phase, and therefore it is crucial to extend our initial design beyond the core, and develop modules to demonstrate how it is possible to integrate for example a MySQL or MongoDB database communication module. The wanted effect is that by starting out with a few modules, which can be adopted for individual use, the popularity would rise, and the open source community would start developing modules that works with their technologies.

Fig. 1 illustrates inversion of control on the modularization of a database connection. An example could be, that a patient measures blood pressure, and wants to report the outcome of the measurement. He would submit the data through his client which would use the HTTP protocol to communicate with the server. Inside the system, a database component would issue an appropriate command, such as a “create” with the patient’s data as input. This command would not result in a direct upload to a database, and this is illustrated in Fig. 1 as the command to a non-existing database. However, when the create command is executed, another command is as well, and this command could be linked to an external database connection module connected to a concrete database, such as a MongoDB NoSQL database, as illustrated in the figure. The database connection module receives the original create command with the patient data embedded, and interpret it, so that the implemented database may understand it.

In conclusion, the inversion of control mechanism and loose coupling could help integration with legacy systems and re-use of modules for new use cases. We realize that this might add to the burden of module designers, but we still recommend the following requirements:

- The system should be designed as a core with modules providing extended functionality. Database handling and implementation of security protocols are modules rather than core functionality.
- The system should implement an inversion of control mechanism to ensure that the core can distribute tasks to loosely coupled modules.

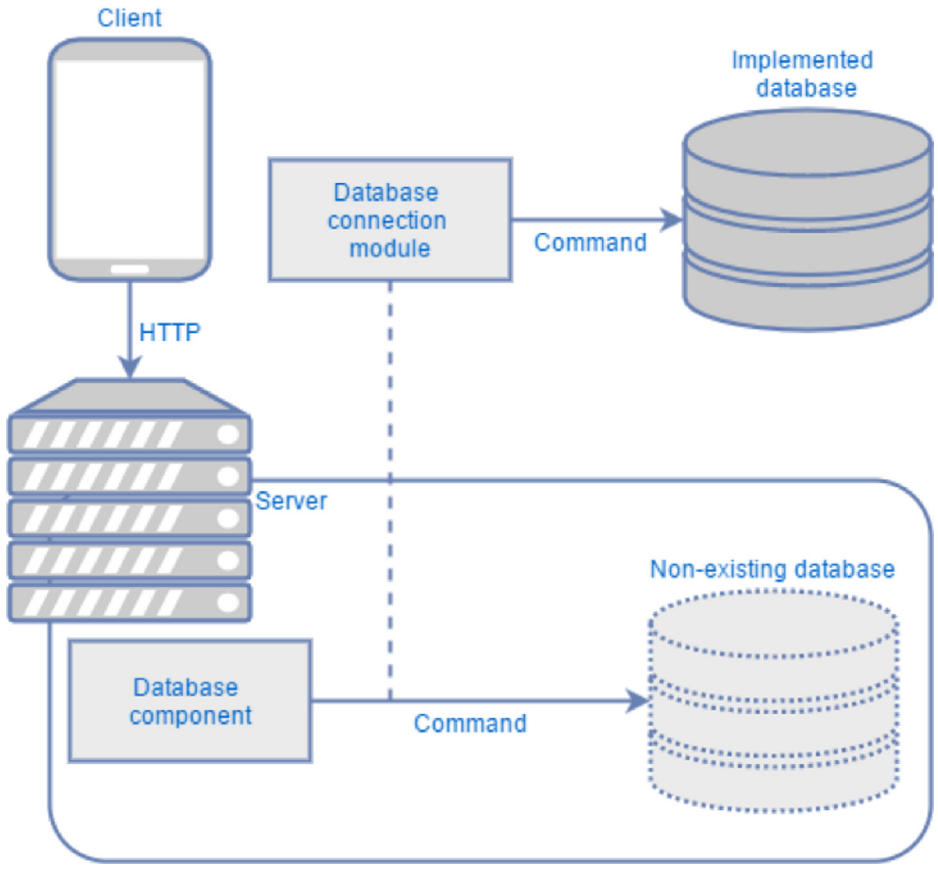

Fig. 1. A visualization of the conceptual inversion of control with a database connection.

2.2. International standards

As documented in the introduction, software re-use in telemedicine is rare. Consequently, each proprietary system is designed to support specific use case – also when it comes to data types and volumes. Solving this problem is both related to sharing actual code and to standardization.

To ensure re-usability of information models and consistent definition of content and validation rules, we wanted our framework to be based on international health information model standards. However, different suitable models exist e.g. openEHR, ISO13606, HL7 CDA and HL7 FHIR [14,15]. We prioritized HL7 FHIR (Fast Interoperable Health Resources) as the primary standard because it is based on web-technologies with a focus on ease of implementation. These characteristics make development of models for telemedicine application easier for developers compared to more specialized and comprehensive models such as openEHR and HL7 v3 models [15]. In addition, HL7 FHIR is built to support RESTfull architectures, a service-oriented paradigm especially suitable for mobile applications because it's protocols are HTTP based i.e. less demanding than e.g. SOAP, and consequently, problems such as short battery life are less likely to occur [16]. These considerations, meant that we favored HL7 FHIR though it is not a fully grown standard because it is currently published as a Standard for Trial Use (STU). Consequently, the standard has been reviewed and approved by HL7 for use in software, but can be subject to change in further versions.

Interoperability with other systems can be obtained if another system already adheres to HL7 FHIR and uses the HTTP protocol. In addition, HL7 FHIR helps bridging the gap to standard-based legacy systems by its ability to map to other HL7 standards [17,18]. For example, a map exists between FHIR and HL7 standard Clinical Document Architecture (CDA), which is currently included the Danish reference architecture for telemedicine applications (in the form of CDA Personal Health Monitoring Report) [19]. However, system based on other standards, or proprietary models will need specialized modules to handle conversion of content to fit into the FHIR profiles.

3. Description of system

The framework that we developed, was named Open Reusable Data System (ORDS). In this section, the intended use of ORDS is described first to explain overall design choices. Afterwards, the architecture is described in more details followed by implementation details and status.

3.1. Intended use: from modules to implementation of telemedicine use cases

In Fig. 2, ORDS' intended use is illustrated. To the left, a community contributes to both the ORDS core, and ORDS modules. At any given point in time, a developer might decide to design an application for a specific telemedicine use case, this is illustrated to the right. The developer might set up an ORDS core of his own, and populate it with modules to support his specific use case. As a minimum, an authorization and a database module is needed because these are designed separate from the core due to the modularization requirements described earlier. If available, these modules can be drawn from, or modified from, the community repository, else they need to be designed for the implementation. More than one database module might be needed if the

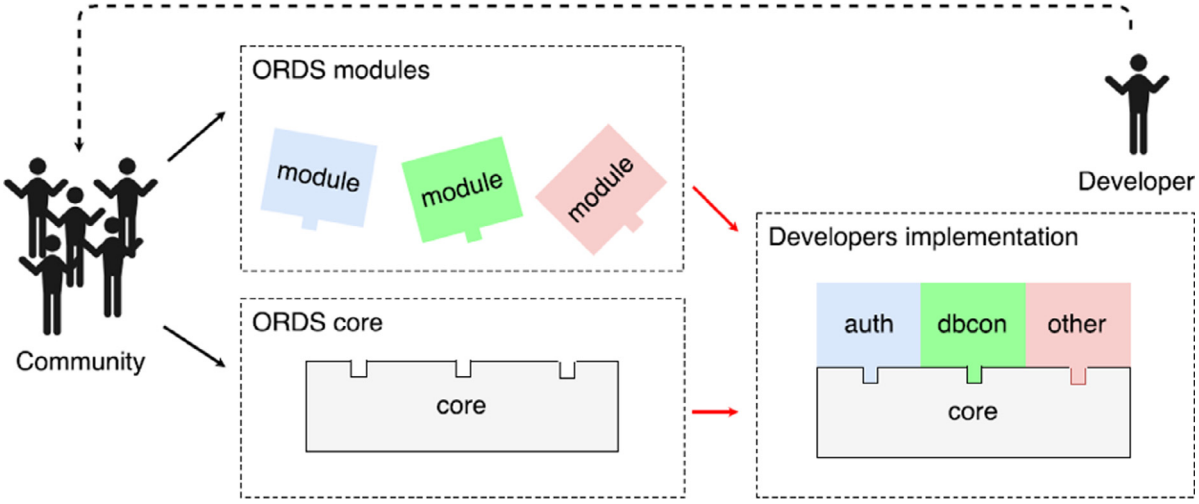

Fig. 2. The concept of the ORDS, showing how a developer can interact with the open source community, and use the functionalities needed in the implementation.

application is intended to bridge legacy systems as well as supporting new functionality. In addition, modules might be needed to support the clinical aspects of the use case, for example the developer might want the server to calculate clinical scores, prepare graphics or perform advanced validation of incoming clinical data instead of handling this on the client side – this functionality can also be added as modules. All new modules and improvement on the core, designed for a specific implementation, can be given back to the ORDS community, in which case the developer also becomes a community member.

3.2. Architecture

ORDS intended use explains the three illustrated repositories of the architecture diagram in Fig. 3. These are the module repository where community constructed modules are situated, the core repository containing the components of the core and the implementation repository which here merely illustrates that each implementation needs a separate configuration component to ensure that the core components work as intended for a given use case. The illustration shows the system architecture as of June 2016, but it may be subject to change due to further development.

The Config component defines initial configurations for a specific implementation, including a whitelist for clients, a limitation for size of a request, and the port on where to run the server. In addition, the implementation specific conformance in relation to the FHIR standard are defined here, such as structure definitions for the resources. Though the use of FHIR is enforced by the core, conformance is placed in the *Config* component because it may be subject to change based on different implementations of ORDS. Combined with a validator, this should ensure conformity between used resources, but it also means that data sent to the databases would contain the mandatory values as a minimum. The *Config* component is implemented as an interface.

The Server component acts as the main application and takes the input of an implementation of the *Config* component. In addition, the *Server* component can distribute its load on several CPU cores to provide vertical scalability. The design means that the Server can instantiate workers corresponding to the number of CPU cores, and that each worker can run ORDS processes.

The *DependencyInjector* component is the implementation of the inversion of control mechanism i.e. it is used to pass references within the core and to modules, to resolve dependencies which thereby leads to a looser coupling. This is done by ensuring that the instances that the *DependencyInjector* component manages and itself only exists as singletons for each worker.

ORDS was also designed to use a hook system. Hooks are functions that can be bound to commands, so that when the commands are executed, so are the hooks. The hooks themselves can change the values of parameters given to the command, and as such, can change the outcome. To administer these hooks the component *HookManager* was designed. The hooks are used in the system to allow the external modules to use functionality through these hooks, where the *DependencyInjector* component provides the needed references.

To allow the flexibility of independency of database technology, the *DBManager* component was designed. This component contains commands following the CRUD (Create, Read, Update, Delete) concepts. However, it is not the responsibility of the *DBManager* to perform these commands with the intention of affecting a database, but instead allow external modules to utilize these functions through hooks. As such, in an actual working implementation, a request from a client to save some data to an implemented database, would invoke a Create command in the *DBManager* instance. Upon the command, a hook is attached from the *HookManager* instance, this would allow an external database connection module (Such as *dbcon* in Fig. 2) to perform an identical Create command, with the provided data, to the implemented database.

To ensure that the data sent to the database is correct, the component *ResourceValidator* was designed. This component compares the data being saved to the resources profiled in the conformance, and checks whether the data type conforms, and if all required fields are populated.

REST was chosen as the interface between ORDS and clients, where requests are send using the HTTP protocol. The *Router* component was used to ensure that the correct function would be executed based on the client's request. *Router* component consists of 3 classes based on the level of interaction; InstanceRoute, TypeRoute, and SystemRoute. These levels of interaction follow the RESTful FHIR API [20].

Between the *Router* functions and the client requests, a *RequestParser* component was defined. This component parses the body of a request, to check for e.g the size of the request to ensure it is not too large, and it also parses queries included in the request. These queries are parsed for use in ORDS, but are also checked for whether they contain illegal search parameters, which should be specified in the FHIR Conformance resource or configuration.

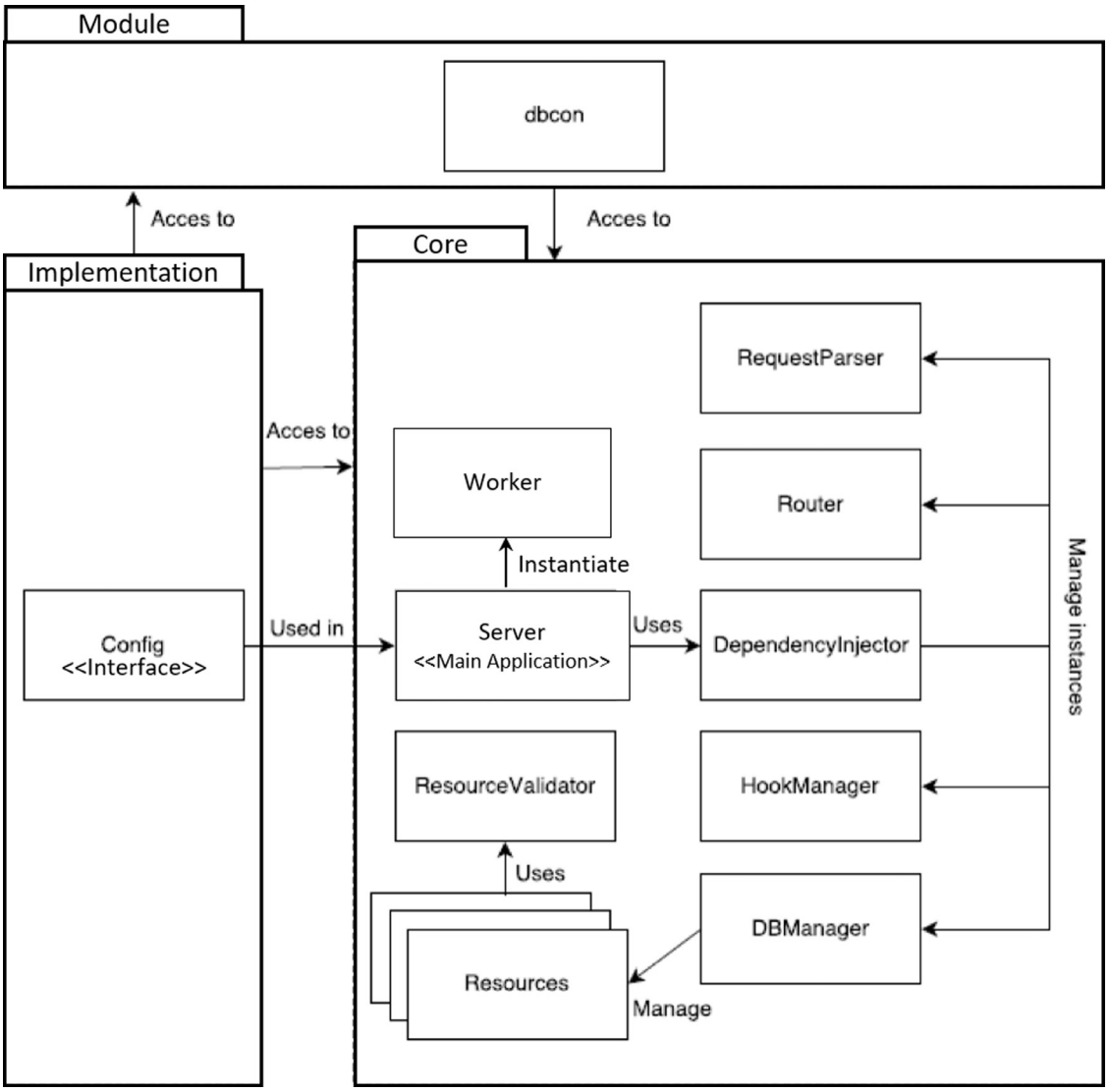

Fig. 3. Illustration of the core artefact where the packages represent different divisions of the system, and the full-drawn boxes illustrate the components. The Server is the main application. The other components are all exposed, so others can utilize them through hooks and/or interfaces. "dbcon" is a module used to illustrate the placement of modules in the architecture.

dbcon is a module used to illustrate the concept and placement of the modules in the architecture. The idea behind a module like "dbcon" is, that different "dbcon" modules would interface to different database technologies, so that one module may ensure a connection to a MSSQL database, while another could ensure connection to a MongoDB database.

3.3. The status of the implementation

ORDS was written using TypeScript as this was found to be a widely known programming language and because of it being more strict and maintainable than JavaScript. TypeScript being easy to learn with a JavaScript background made the potential community even larger. The Express framework [21] was used to assist with the routing. FHIR is not a fully grown standard, and was used in the Draft Standard for Trial Use 2 (DSTU2) which means that the standard has been reviewed and approved by HL7 for use in software, but can be subject to change in further versions (Which have already happened with STU3).

HookManagement, ResourceValidation and *DependencyInjection* have each been split into their own module, and each is fully functional. This is done since they can function as standalone software without the rest of the ORDS software. All functionality from the design have now been implemented, and each module have been tested. However, no automatic test has been written for the code yet, and end-to-end testing will follow our first clinical implementation cases.

The code is distributed on https://github.com/ords with an MIT license and the functioning packages are distributed on https://www.npmjs.com/package/@ords/core. This link contains all the modules developed by the project group. Modules developed by the project group and/or third party that do manipulations of the core directly would be available from https://www.npmjs.com/browse/depended/ords-fhir. The MIT license allows for full participation of the community with ORDS but limits warranty and ensures copyright of ORDS. Others might therefore use and develop ORDS but they cannot claim it as their own.

During the implementation, the original design of ORDS have only been subject to minor changes. Designing a highly modular system like ORDS that makes use of hooks rather than already compiled functions, will inevitably affect the performance of the system regarding throughput. The strength of ORDS lies with its

modularity. However, some rigidity is introduced in our specification of what hooks are available within the core. As such, ORDS represents a certain tradeoff between how much rigidity and flexibility it provides.

4. Discussion

In our study, we developed and did a pilot implementation of an architecture for telemedicine where we balanced the need for standardization, re-use and interoperability, with the need for adaptable solutions which can integrate with new functionalities as well as legacy systems. We made our architecture approach modular and open source, so that software components can be reused and we based our architecture on the international information model standard, HL7 FHIR, to ensure information interoperability. The modular approach with its inversion of control mechanisms also makes it possible to use ORDS core functionality using different databases, other standards, or legacy systems because the loose coupling makes the architecture highly adaptable. We evaluated the technical feasibility of our architecture in the implementation, which can be found on github. Our study suggests that technically, there is no hindrance in building highly adaptable and yet interoperable core frameworks for telemedicine.

When we compare our solution with other health IT architectures, we see that others have suggested before us to have a standardized information model deeply integrated in the system architecture [22]. However, we have not seen others that did not let that choice influence database design. What is more common is that one chosen database implements a schema based on the standards reference information model, given that the standard is based on a dual modeling approach. For example, Austin [22] choose to implement a PostgreSQL database based on 13,606 RM, because the systems content management is based on 13,606 archetypes. In ORDS, we chose that database handling should be regarded yet another module, and this gives extended flexibility of our solution. On the other hand, this also means that the complexity of supporting a new telemedicine use case in ORDS increases, compared to supporting a new archetype or template in Austin's system, simply because new projects must spend time on choosing and possibly designing a database. Which design approach is more appropriate is probably very dependent on context, and our choice have been based on the requirement of being able to integrate with legacy systems, in which case the flexibility in database management is very beneficial.

Looking at trends for telemedicine solutions, it is apparent that telemedicine systems become increasingly more advanced e.g. with build-in decision support [11], but the architectural considerations are still limited, which means that even though a cloud based system such as Melillo et al's [11] is scalable, it is neither modular, standardized or open source which means that re-use and interoperability becomes a big challenge. Even in research that summarize what is currently known about telemedicine architecture design, the focus is mainly on user comfort, security/privacy, the ability of systems to work despite offline periods, and scalability [23]. However, in the context of the Continua initiative [24], interoperability of personalized mobile health technologies is a priority, and the strategy chosen for interoperability is largely based on HL7 and IHE standards and profiles e.g. HL7 CDA QFD is used for representing questionnaires. Franz et al [16] have suggested that Continua/IHE based solutions might be challenged by low battery life of smart phones due to high data traffic because they use a classic SOAP-based architecture. Consequently, Franz et al suggested and tested a more lightweight solution based on a based on a HL7 FHIR RESTfull architecture, and saw a significant decrease in data traffic. From the above examples, we can see that the R&D initiatives in telemedicine is fragmented. On the one hand, most clinical solutions reported on does not consider interoperability and re-use, but focus on improving clinical outcomes e.g. by implementing decision support. On the other hand, interoperability in telemedicine is a priority in the Continua initiative, and among interoperability oriented researchers. However, bridging the gap is a challenge. One way of finding common ground, would be to ensure that interoperability initiatives, in accordance with our solution, put special emphasis on the integration with legacy systems. However, we realize that the technicalities are only the top of the iceberg of a difficult transition towards clinically efficient, yet interoperable telemedicine solutions that needs to take place in the years to come.

Another key aspect of our study has been to develop ORDS as an open source project. Our vision is that an open source community committed to a certain framework will enhance development speed, because modules can be shared and re-used. Danish initiatives have been directed towards forming open source communities for telemedicine [19]. However, with a small population building a community of developers interested in telemedicine, open source and standardized infrastructure have not been successful. We acknowledge, that building a community is about spread of innovation, which is intangible and difficult to control. However, once a community is forming community participation management aspects should be given special attention. Previous open source projects have implemented coding guidance and fixed repository structures, as well as forum to interact with other developers e.g. https://wordpress.com/, https://www.meteor.com/ and https://www.npmjs.com/. In contrast to many open source projects, where a structured documentation strategy is not always formed, Johansen and Löfgren [25] have suggested that focus on simple documentation by collecting documentation in one place, and using references rather than copy paste would ensure that changes in the system does not lead to inconsistent documentation. It will be one of our future priorities to build a community participation management strategy with special attention on documentation.

In conclusion, our study showed that highly adaptable and yet interoperable core frameworks for telemedicine can be build. However, technical feasibility does not guarantee success, and several tasks are still unresolved. Clinical demonstration and the challenges of building an open source community are the most notable future challenges.

For future work, since our study did not evaluate the feasibility of building a telemedicine solution for a specific use case nor did we implement or test such a solution in clinical practice. This is the logical next step in our work

Conflict of interest

The authors have no conflicts of interest to declare

References

[1] R. Busse, L. Blümel, D. Scheller-Kreinsen, Tackling Chronic Disease in Europe: Strategies, Interventions and Challenges, World Health Organization, Albany, 2010.
[2] G. Graffigna, S. Barello, G. Riva, A.C. Bosio, Patient engagement: the key to redesign the exchange between the demand and supply for healthcare in the era of active ageing, in: Active Ageing and Healthy Living: A Human Centered Approach in Research and Innovation as Source of Quality of Life, 203, 2014, pp. 85–95.
[3] N. van den Berg, M. Schumann, K. Kraft, W. Hoffmann, Telemedicine and telecare for older patients—a systematic review, Maturitas 73 (2) (2012) 94–114.
[4] S. Koch, Home telehealth—Current state and future trends, Int. J. Med. Inform. 75 (8) (2006) 565–576.
[5] P. Zanaboni, R. Wootton, Adoption of telemedicine: from pilot stage to routine delivery, BMC Med. Inf. Decis. Making 12 (1) (2012) 1.
[6] A.W. Charlotte, T. Pamela, Rapid EHR development and implementation using web and cloud-based architecture in a large home health and hospice organization, Nurs. Inform. (2014).

[7] M.W. Bridges, SOA in healthcare, Sharing system resources while enhancing interoperability within and between healthcare organizations with service-oriented architecture, Health Manage. Technol. 28 (6) (2007) 10 Jun.
[8] A. Lee, M. Sandvei, K.R. Christiansen, J. Petersen, T. Hosbond, Klinisk Integreret Hjemmemonitorering (KIH) Slutrapportering til Fonden for Velfærdsteknologi, 2017 Available at https://www.digst.dk/~/media/Files/Velfaerdsteknologi/Telemedicin/11d-projekter/KIH-Slutrapport.pdf Accessed 28/12/.
[9] L. van Velsen, J. Solana, W. Oude-Nijeweme D'Hollosy, F. Garate-Barreiro, M. Vollenbroek-Hutten, Advancing telemedicine services for the aging population: the challenge of interoperability, Stud. Health Technol. Inform. 217 (2015) 897–900.
[10] A.R. Hardisty, S.C. Peirce, A. Preece, C.E. Bolton, E.C. Conley, W.A. Gray, et al., Bridging two translation gaps: a new informatics research agenda for telemonitoring of chronic disease, Int. J. Med. Inf. 80 (10) (2011) 734–744.
[11] P. Melillo, A. Orrico, P. Scala, F. Crispino, L. Pecchia, Cloud-based smart health monitoring system for automatic cardiovascular and fall risk assessment in hypertensive patients, J. Med. Syst. 39 (10) (2015) 1–7.
[12] T. Teijeiro, P. Félix, J. Presedo, C. Zamarrón, An open platform for the protocolization of home medical supervision, Expert Syst. Appl. 40 (7) (2013) 2607–2614.
[13] M. Fowler, Inversion of Control Containers and the Dependency Injection Pattern, 2004 Available at http://martinfowler.com/articles/injection.html Accessed 07/13, 2016.
[14] P. Knaup, O. Bott, C. Kohl, C. Lovis, S. Garde, Electronic patient records: moving from islands and bridges towards electronic health records for continuity of care, Yearb Med. Inform. (2007) 34–46.
[15] HL7, FHIR Homepage, 2017 Available athttps://www.hl7.org/fhir/.
[16] B. Franz, A. Schuler, O. Krauss, Applying FHIR in an integrated health monitoring system, EJBI 11 (2) (2015).
[17] Relationship between FHIR and v3 Messaging - FHIR v0.0.81. Available at: http://www.hl7.org/implement/standards/fhir/comparison-v3.html Accessed 8/13/2014.
[18] Relationship between FHIR and CDA - FHIR v0.0.81. Available at: http://www.hl7.org/implement/standards/fhir/comparison-cda.html Accessed 8/13/2014
[19] National eHealth Authority, Reference Architecture for Collecting Health Data from Citizens, 2013 Available at https://sundhedsdatastyrelsen.dk/-/media/sds/filer/rammer-og-retningslinjer/referenceaktitektur-og-it-standarder/referencearkitektur/referencearchitecture-collecting-health-data-citizens.pdf Accessed 28/12/, 2017.
[20] HL7, FHIR RESTful API, 2017 Available at https://www.hl7.org/fhir/http.html Accessed 28/12/, 2017.
[21] Node.js Foundation, Express, 2017 Available at https://expressjs.com/ Accessed 28/12/, 2017.
[22] T. Austin, S. Sun, Y.S. Lim, D. Nguyen, N. Lea, A. Tapuria, et al., An electronic healthcare record server implemented in PostgreSQL, J. Healthcare Eng. 6 (3) (2015) 325–344.
[23] M.M. Baig, H. Gholamhosseini, Smart health monitoring systems: an overview of design and modeling, J. Med. Syst. 37 (2) (2013) 1–14.
[24] PCH Alliance, 2016 Continua Design Guideline, 2016 Available at https://cw.continuaalliance.org/document/dl/14699 Accessed 28/12/, 2017.
[25] N. Johansson, A. Löfgren, Designing for Extensibility: An action Research Study of Maximizing Extensibility By Means of Design Principles, University of Gothenberg, 2009.